

ASTEROID FIELD

Programma sviluppato in Three.js che implementa l'orbita circolare di alcuni asteroidi nello spazio con una navicella spaziale a sua volta in orbita attorno ad essi. Sono implementate funzionalità come collisioni tra asteroidi, esplosioni, possibilità di spostare asteroidi dall'orbita originale.

1. INIZIALIZZAZIONE SCENA

Il renderer WebGL viene creato e aggiunto al DOM. Scena e camera prospettica vengono istanziate, poi OrbitControls con damping per un'inerzia fluida su orbiting/pan/zoom.

Le luci sono due: una AmbientLight calda come fill generale e una DirectionalLight come sole principale.

La GUI lil-gui espone i parametri modificabili dall'utente raggruppati in cartelle: velocità orbite e spin, raggio e spessore del campo asteroidi, intensità luci, toggle collisioni, parametri esplosioni (conteggio particelle, durata, velocità, dimensione) e parametri navicella.

Infine vengono definiti il path della skybox e i sei nomi file della cubemap (px/nx/py/ny/pz/nz), pronti per essere caricati nel passo successivo.

2. SPAWN ASTEROIDI E NAVICELLA

Si definiscono le impostazioni del campo: 20 asteroidi, raggio 200, altezza 100 e dimensione casuale tra 5 e 25. Poi si crea un contenitore (Group) che raccoglierà tutti gli asteroidi. Quando il modello dell'asteroide è pronto, viene ridimensionato normalizzando la grandezza originale del file.

Si crea quindi ogni asteroide: per ognuno dei 20 elementi si fa una copia del modello originale, lo si mette in una posizione casuale sull'orbita e gli si assegna una dimensione casuale. Dopo aver applicato la scala, si calcola la bounding sphere che contiene l'asteroide (per ottenere un raggio corretto utile per i controlli di collisione). Poi viene assegnata una rotazione iniziale casuale.

Nei dati personali dell'asteroide (userData) vengono salvate tutte le informazioni che servono durante l'animazione. Vengono salvati anche i valori iniziali della distanza e dell'altezza per poter modificare le orbite dalla grafica di controllo senza perdere i valori originali.

Infine i materiali vengono copiati separatamente, perché normalmente le copie condividono lo stesso materiale. In questo modo modificare l'aspetto di un asteroide non cambia tutti gli altri.

Successivamente viene caricato il modello della navicella, ridimensionato e adattato. Si attivano le ombre su tutte le sue parti, viene posizionata nella sua posizione iniziale, si calcola la sfera per le collisioni e infine viene aggiunta alla scena.

3. FUNZIONI IMPLEMENTATE

- **findRootAsteroid**: Risale la gerarchia di nodi finché non trova il figlio diretto di asteroidGroup. Serve perché il raycaster colpisce il mesh foglia, ma drag e orbita devono agire sul clone radice.

- **updateHoverHighlight**: Ogni frame lancia un raggio dal cursore verso la scena. Se colpisce un asteroide ne salva l'emissive originale e lo imposta rosso; quando il cursore si sposta ripristina il colore precedente.

- **checkCollisions**: Test sphere-sphere tra tutte le coppie di asteroidi. Se la distanza tra i centri è minore della somma dei raggi, crea un'esplosione al punto medio e aggiunge entrambi a un Set di rimozione. La rimozione avviene dopo il loop per non invalidare l'iterazione.

- **createExplosion**: Crea un BufferGeometry con tutte le particelle al punto dell'esplosione, ognuna con una velocità casuale sulla sfera unitaria. Aggiunge una PointLight temporanea e un oggetto Points alla scena, poi li inserisce nell'array explosions per l'aggiornamento frame-by-frame.

- **updateShip**: Avanza l'angolo orbitale e aggiorna posizione e orientamento della navicella. Controlla collisioni con ogni asteroide: se ne trova una crea un'esplosione e imposta shipDead.

4. EVENT HANDLER

Tre listener gestiscono il drag degli asteroidi.

- **pointerdown**: trasforma il click in coordinate Three.js, crea un raggio e controlla se colpisce un asteroide. Se sì, avvia il trascinamento creando un piano orientato verso la telecamera e passante per il punto del click.
- **pointermove**: aggiorna sempre il puntatore in coordinate Three.js per l'evidenziazione. Calcola il punto sul piano di movimento, applica l'offset e converte la posizione nello spazio locale del gruppo prima di spostare l'asteroide
- **pointerup**: ricalcola i dati dell'orbita dalla nuova posizione dell'asteroide dopo il rilascio. Aggiorna la distanza dal centro, l'angolo e l'altezza. Poi chiude il trascinamento, ripristina i controlli della camera dopo un frame per evitare che il rilascio venga interpretato come un nuovo comando di rotazione.

5. ANIMATE

Ogni frame aggiorna l'highlight hover, poi sposta ogni asteroide lungo la sua orbita usando un quaternion e lo fa ruotare su se stesso (gli asteroidi in drag vengono saltati). Controlla le collisioni e aggiorna la navicella. Per ogni esplosione attiva muove le particelle integrando $posizione \times velocità \times dt$, sfuma opacità e luce linearmente, e quando la vita scende a zero rimuove tutto dalla scena liberando la memoria GPU. Infine aggiorna OrbitControls e renderizza.